

Name: Y.Anjaneyulu

Reg.No.: 192412355

Q. End-to-End 5G Multirate Receiver Simulation

Simulate an end-to-end 5G receiver employing multi-rate DSP techniques and evaluate signal quality. (25) (BL5) (WK4)

Given Data:

- Input Sampling Rate = 245.76 MHz
- Decimation Factor = 8
- OFDM Signal Bandwidth = 100 MHz

Tasks:

1. Generate the received signal.
2. Implement filtering.
3. Perform decimation.
4. Estimate output spectrum.
5. Evaluate signal quality.
6. Analyze processing efficiency.

Code :

```
%% End-to-End 5G Multirate Receiver Simulation
```

```
% Parameters
```

```
Fs_in = 245.76e6; % Input sampling rate (Hz)
```

```
M = 8; % Decimation factor
```

```
BW = 100e6; % OFDM signal bandwidth (Hz)
```

```
Fs_out = Fs_in / M; % Output sampling rate after decimation
```

```
N = 2^16; % Number of samples for simulation
```

```
%% 1. Generate the received OFDM-like signal
```

```

t = (0:N-1)/Fs_in;

% Simulate a bandlimited OFDM-like signal using random QAM symbols
numCarriers = 1024;
data = randi([0 3], numCarriers, 1); % QPSK symbols
qamSig = qammod(data, 4);

% Place carriers in frequency domain
X = zeros(N,1);
X(1:numCarriers) = qamSig;
rxSig = ifft(X); % Time-domain OFDM-like signal

% Add noise
rxSig = rxSig + 0.01*randn(size(rxSig));

%% 2. Filtering (anti-aliasing filter before decimation)
% Design low-pass filter with cutoff = BW/2
lpFilt = designfilt('lowpassfir', ...
    'PassbandFrequency', BW/2/(Fs_in/2), ...
    'StopbandFrequency', (BW/2+20e6)/(Fs_in/2), ...
    'PassbandRipple', 0.1, ...
    'StopbandAttenuation', 60, ...
    'DesignMethod', 'equiripple');

filteredSig = filter(lpFilt, rxSig);

%% 3. Perform decimation
decSig = downsample(filteredSig, M);

%% 4. Estimate output spectrum

```

```

figure;
pwelch(decSig, hamming(2048), 1024, 4096, Fs_out);
title('Output Spectrum after Decimation');

%% 5. Evaluate signal quality (SNR)
signalPower = bandpower(decSig);
noisePower = bandpower(decSig - rxSig(1:length(decSig)));
SNR = 10*log10(signalPower/noisePower);
fprintf('Estimated SNR after processing: %.2f dB\n', SNR);

%% 6. Analyze processing efficiency
fprintf('Input samples: %d\n', length(rxSig));
fprintf('Output samples after decimation: %d\n', length(decSig));
fprintf('Reduction factor: %.2f\n', length(rxSig)/length(decSig));

```

Output:

